

Learn Low-Level Design in 25 Days: Day 7 — Designing a Parking Lot System (Step-by-Step…

The Problem Every Engineer Faces: “Where Do I Start?”

TechPreneur

Published October 19, 2025

Free: No

Read on Freedium · original

Contents

Step 1: Understanding the Problem 0

Step 2: Identify Core Objects 0

Step 3: Rough Class Diagram 0

Step 4: Relationships and Responsibilities 0

Step 5: Translating to Code (Simplified) 0

Step 6: Design Considerations & Extensions 0

Try It Yourself 0

Key Takeaways 0

What's Next? 0

Every developer remembers their first system design interview question that went, "Design a Parking Lot System." It sounds harmless — like something you could solve in 15 minutes. Then, two minutes later, you're already drowning in questions.

- What types of vehicles do we support?
- How do we handle tickets, payments, or full capacity?
- Do we have multiple floors or entrances?

That's when you realize: even a parking lot is a miniature ecosystem of interacting objects, business rules, and constraints — the perfect problem to practice real-world Low-Level Design.



Welcome to **Day 7 of Learn Low-Level Design in 25 Days** — today, we're going hands-on to **design a Parking Lot System step-by-step**. By the end of this walkthrough, you'll know how to:

- Break down ambiguous requirements into objects and relationships
- Model classes and responsibilities cleanly
- Apply core OOP principles (composition, inheritance, encapsulation)
- Translate UML diagrams into real, maintainable code

This is where your LLD foundation starts turning into real engineering intuition.

Step 1: Understanding the Problem

Before writing a single class, let's clarify the scope — a trap many candidates skip.

Requirements:

1. The system should manage parking for different vehicle types — cars, bikes, and trucks.
2. Each vehicle gets a ticket when entering.
3. The system should track parking slots, entry/exit gates, and payments.
4. When the lot is full, no new vehicle should be allowed in.

We'll assume this is for a **multi-level parking lot** in a shopping mall — not a smart IoT garage. (That would be Day 20 material!)

Step 2: Identify Core Objects

Let's "think in objects." What are the main nouns here?

- **ParkingLot** — top-level entity that manages levels, entry gates, and exits.
- **ParkingFloor** — each level with multiple slots.
- **ParkingSlot** — an individual spot that can hold one vehicle.
- **Vehicle** — base class for Car, Bike, Truck.
- **Ticket** — represents parking access and tracks entry/exit time.
- **Payment** — handles billing when a vehicle exits.

Now we'll define responsibilities and relationships.

Step 3: Rough Class Diagram

```
ParkingLot
 ├── List<ParkingFloor>
 ├── List<EntryGate>
 ├── List<ExitGate>
 ├── assignSlot(Vehicle)
 └── freeSlot(Ticket)

ParkingFloor
 ├── int floorNumber
 ├── Map<VehicleType, List<ParkingSlot>>
 └── getAvailableSlot(VehicleType)

ParkingSlot
 ├── String slotId
 ├── VehicleType type
 ├── boolean isOccupied
 └── parkVehicle(Vehicle)

Vehicle
 ├── String licensePlate
 ├── VehicleType type

Ticket
 ├── String ticketId
 ├── Vehicle vehicle
 ├── ParkingSlot slot
 ├── LocalDateTime entryTime
 └── LocalDateTime exitTime

Payment
 ├── Ticket ticket
 ├── double amount
 └── processPayment()
```

This gives us a clean, modular view of the system.

Step 4: Relationships and Responsibilities

Let's zoom in on the core relationships:

- **ParkingLot** has multiple **ParkingFloors**.
- **ParkingFloor** manages a collection of **ParkingSlots** categorized by **VehicleType**.
- **EntryGate** issues a **Ticket** and assigns a slot.
- **ExitGate** processes **Payment** and frees the slot.
- **Vehicle** interacts only with **Gate** — never directly with floors or slots (encapsulation!).

Think of it like a real-world airport — you don't assign your own gate; the system does it for you.

Step 5: Translating to Code (Simplified)

Here's a simplified Java-like design for clarity:

```
enum VehicleType { CAR, BIKE, TRUCK }

class Vehicle {
    String licensePlate;
    VehicleType type;
}

class ParkingSlot {
    String id;
    VehicleType type;
    boolean isOccupied = false;
    Vehicle parkedVehicle;

    boolean park(Vehicle v) {
        if (isOccupied || v.type != this.type) return false;
        this.parkedVehicle = v;
        this.isOccupied = true;
        return true;
    }

    void free() {
        this.parkedVehicle = null;
        this.isOccupied = false;
    }
}

class Ticket {
    String ticketId;
    Vehicle vehicle;
    ParkingSlot slot;
    LocalDateTime entryTime;
}

class ParkingFloor {
    int floorNumber;
    Map<VehicleType, List<ParkingSlot>> slots;

    ParkingSlot getAvailableSlot(VehicleType type) {
        for (ParkingSlot s : slots.get(type))
            if (!s.isOccupied) return s;
        return null;
    }
}

class ParkingLot {
    List<ParkingFloor> floors;

    Ticket assignSlot(Vehicle v) {
        for (ParkingFloor f : floors) {
            ParkingSlot s = f.getAvailableSlot(v.type);
            if (s != null) {
                s.park(v);
```

```
                return new Ticket(UUID.randomUUID().toString(), v, s, LocalDateTime.now())
;
            }
        }
        throw new RuntimeException("No slots available");
    }
}
```

Step 6: Design Considerations & Extensions

Once the skeleton is ready, we can evolve the design:

- **Scalability**: Add a `SlotAllocationStrategy` (Strategy Pattern) to handle different allocation rules (nearest-first, random, VIP).
- **Concurrency**: Make slot allocation thread-safe for multiple entry gates.
- **Persistence**: Add repository layers to store ticket and payment data.
- **Extensibility**: Introduce `ElectricCarSlot` or `HandicappedSlot` subclasses when business needs evolve.

That's the beauty of good design — **you can grow without breaking**.

Try It Yourself

1. Add Payments: Design a simple `PaymentProcessor` class that calculates cost based on parking duration (e.g., ₹20/hour).

2. Handle Multiple Entrances: What if there are 3 entry gates? How would you synchronize slot allocation across them safely?

3. Add a Display Board: Design a `DisplayBoard` class that shows available slots per floor and updates dynamically.

Key Takeaways

- Always start by clarifying **requirements and scope**.
- Identify **core entities** (nouns) and their **behaviors** (verbs).
- Model relationships using **composition** before inheritance.
- Build for **extensibility** — real systems always evolve.
- Code should reflect reality: **each object does one thing well**.

What's Next?

Tomorrow, in **Day 8 — LLD Patterns Every Engineer Must Know: Singleton, Factory, and Builder**, we'll dive into **object creation patterns** — the tools that make systems like this flexible and maintainable at scale.

You can follow or bookmark the full **"Learn Low-Level Design in 25 Days"** series here: https://techpreneurr.medium.com/list/lowlevel-design-in-25-days-a-practical-bootcamp-for-devs-8829ef3508d8 (https://freedium-mirror.cfd/https://techpreneurr.medium.com/list/lowlevel-design-in-25-days-a-practical-bootcamp-for-devs-8829ef3508d8)